Code Review Report — Project Pegasus Sprint 3

Project: Pegasus BaaS Platform **Date:** 2026-04-09 **Reviewer:** Engineering Lead **Review Period:** Mar 31 — Apr 9, 2026 **Commits Reviewed:** 500+ across 20 repositories

---

1. Review Summary

Area	Rating	Evidence
Architecture	**A**	Clean microservices separation, 16 services with clear domain boundaries. YARP proxy for partner traffic, Ocelot for admin. Event-driven via Kafka.
Security	**B+**	Argon2id key hashing, HMAC-SHA256 signing, PII scrubbing, WORM audit logs. Gaps: missing security headers (PEG-377), catalog publicly accessible (PEG-378).
Code Quality	**B+**	Consistent patterns across services (.NET 9/10, Clean Architecture in Banking API/User Management). FluentValidation for inputs. RFC 7807 error format (mostly). Gap: inconsistent error formats between Admin Auth and UMS (PEG-387).
Testing	**B**	566 test cases documented, 183 executed via Playwright. k6 load test suite written but not executed. No unit tests found in most services.
CI/CD	**A**	Full Docker Hub pipeline for all 16 services. Slack notifications. CODEOWNERS for auto-review. Secrets stripped from config.
Documentation	**A+**	55+ Confluence pages, complete OpenAPI spec (22 endpoints), Mintlify developer docs with Try It playground. Best-in-class for Nigerian fintech.
DevOps	**B**	Successful VPS migration. But: 11/16 dev services down. No centralized logging. No auto-recovery.

Overall Grade: B+

---

2. Key Code Review Findings

2.1 Positive Findings

1. PII Scrubbing Is Comprehensive

- `PiiScrubberDestructuringPolicy` covers 20+ field names across all services
- Fields masked: BVN, NIN, account numbers, passwords, tokens, phone numbers, date of birth, card numbers, CVV, PIN, and more
- Applied at the Serilog sink level — no developer opt-in required, scrubbing happens automatically

2. Idempotency Pattern Correctly Implemented

- 24-hour Redis TTL for idempotency keys
- `X-Idempotency-Replayed: true` header set on duplicate requests
- Original response body cached and replayed — correct behavior for financial APIs

3. Circuit Breaker Uses Redis-Backed State Machine

- State transitions: `CLOSED` → `OPEN` → `HALF_OPEN` → `CLOSED`
- Redis-backed state ensures consistency across multiple service instances
- Correct distributed design — avoids per-instance circuit breaker drift

4. WORM Audit Logs

- `INSERT`-only table with `DENY UPDATE` and `DENY DELETE` triggers at the database level

- Provides tamper-evident audit trail — good compliance foundation for CBN requirements
- Each entry includes: actor, action, resource, timestamp, IP address, request/response hashes

5. Sandbox Isolation Is Strong

- API keys prefixed with `ubn_sb_` are routed exclusively to sandbox services
- Sandbox keys **never** reach real banking infrastructure
- Separate database schemas for sandbox data — no risk of production data contamination

6. Token Bucket Rate Limiting Uses Atomic Lua Script

- Redis Lua script executes bucket check and decrement atomically
- Prevents race conditions under concurrent requests
- Correctly returns `429 Too Many Requests` with `Retry-After` header

7. Webhook Retry Schedule Is Production-Grade

- Retry intervals: immediate → 1 minute → 5 minutes → 30 minutes → 2 hours → 24 hours
- After all retries exhausted, failed webhooks are sent to dead-letter queue with SIEM integration
- Each retry attempt is logged with response status and latency

---

2.2 Concerns and Issues

1. No Unit Tests in Most Backend Services

- **Severity:** Medium
- **Details:** Only integration/E2E tests exist via Playwright. No xUnit/NUnit test projects found in most .NET services.
- **Risk:** Regressions cannot be caught early. Refactoring is risky without unit test safety net.
- **Ticket:** Recommend creating PEG-400

2. Login Returns 200 for Failures

- **Severity:** Medium
- **Details:** `POST /api/Auth/login` returns HTTP 200 with `{"isSuccess": false, "responseCode": "999"}` for invalid credentials. This breaks REST conventions and confuses API consumers who check HTTP status codes.
- **Risk:** Partner integrations may not handle this correctly. Monitoring tools will not flag these as errors.
- **Ticket:** PEG-387 (related)

3. mTLS Not Bypassed in Sandbox

- **Severity:** High
- **Details:** Sandbox payment endpoints require mTLS certificates, same as production. Partners cannot test payment flows in sandbox without obtaining certificates first.
- **Risk:** Blocks sandbox adoption entirely for payment testing.
- **Ticket:** Recommend creating PEG-401

4. Admin Auth Doesn't Use RFC 7807

- **Severity:** Low
- **Details:** Admin Auth service returns custom error objects `{"isSuccess": false, "responseCode": "..."}` instead of RFC 7807 problem details `{"type": "...", "title": "...", "status": ..., "detail": "..."}`. Other services (User Management, Banking API) correctly use RFC 7807.
- **Risk:** Inconsistent developer experience, harder to write generic error handlers.
- **Ticket:** PEG-387

5. Commented-Out RedisRepository.cs in User Management

- **Severity:** Low
- **Details:** `RedisRepository.cs` in the User Management service has large blocks of commented-out code, suggesting incomplete Redis caching work.

- **Risk:** Indicates unfinished feature. If Redis caching was intended for session management, its absence could impact performance at scale.
- **Ticket:** Recommend backlog item

6. DemoAdAuthClient Hardcoded

- **Severity:** Low
- **Details:** The `DemoAdAuthClient` implementation is directly instantiated with no abstraction layer. No interface-based DI registration that would allow easy swap to a real Active Directory client.
- **Risk:** When integrating with real AD/LDAP, significant refactoring will be required.
- **Ticket:** Recommend backlog item

7. No Database Connection Pooling Limits Specified

- **Severity:** Medium
- **Details:** SQL connection strings across services do not specify `MaxPoolSize`, `MinPoolSize`, or `ConnectionLifetime` parameters. Default .NET pool size is 100, which may be insufficient or excessive depending on the service.
- **Risk:** Could cause connection exhaustion under load or waste resources with idle connections.
- **Ticket:** Recommend creating PEG-402

8. Kafka Consumer Offset Commits After DB Write

- **Severity:** Low
- **Details:** Kafka consumers commit offsets after writing to the database. This is the correct "at-least-once" pattern. However, there is no idempotency check on the consumer side — if a message is reprocessed after a crash, duplicate records could be created.
- **Risk:** Duplicate transaction records in edge cases (crash between DB write and offset commit).
- **Ticket:** Recommend backlog item

---

3. PR Review Activity (This Sprint)

Metric	Value
PRs Merged	47 (across development/main branches in all repos)
Primary Reviewers	Tomachi (GitHub), Obioma Chijindu Ezekiel
Review Turnaround	Most PRs merged same day
Branch Strategy	Feature branches → `development` → `main`
Auto-Review	CODEOWNERS file configured for automatic reviewer assignment

PR Distribution by Repository Area

Area	PRs
Partner Portal (Frontend)	12
Admin Portal (Frontend)	8
Gateway Services	6
Banking API	5
User Management	5
Infrastructure / DevOps	4
Documentation	4

Other Services	3

---

4. Recommendations

Priority 1 — Must Address

1. **Add unit test projects to all .NET services** — minimum coverage: service layer methods, FluentValidation validator tests, and utility/helper classes. Target: at least one test project per service by Sprint 4.
2. **Standardize error responses across ALL services to RFC 7807** — Admin Auth is the primary offender. Create a shared NuGet package with standard error response middleware.
3. **Fix login endpoint to return 401 for authentication failures** — this is a breaking change and must be coordinated with the frontend team and any existing API consumers. Publish migration guide.

Priority 2 — Should Address

4. **Add connection pool limits to all SQL connection strings** — recommend `MaxPoolSize=100;MinPoolSize=5;Connection Lifetime=300` as defaults, tuned per service based on expected load.
5. **Implement structured code review checklist** covering:
   - Security (input validation, auth checks, secrets handling)
   - Performance (N+1 queries, unnecessary allocations, async/await correctness)
   - Error handling (proper HTTP status codes, RFC 7807 compliance, error logging)
   - Testing (unit tests for new logic, integration test updates)
6. **Add pre-commit hooks for secrets scanning** — integrate Gitleaks or similar tool to prevent accidental credential commits. Configure as both pre-commit hook and CI pipeline step.

Priority 3 — Nice to Have

7. **Set up SonarQube or similar for code quality metrics** — track code coverage, code smells, technical debt, and security hotspots across all repositories. Integrate with PR checks to enforce quality gates.

---

5. Sign-Off

Role	Name	Signature	Date
Engineering Lead			
CTO			

Document generated: 2026-04-09 Classification: Internal — Project Pegasus